

ROS Documentation



Best works on Ubuntu 16.04

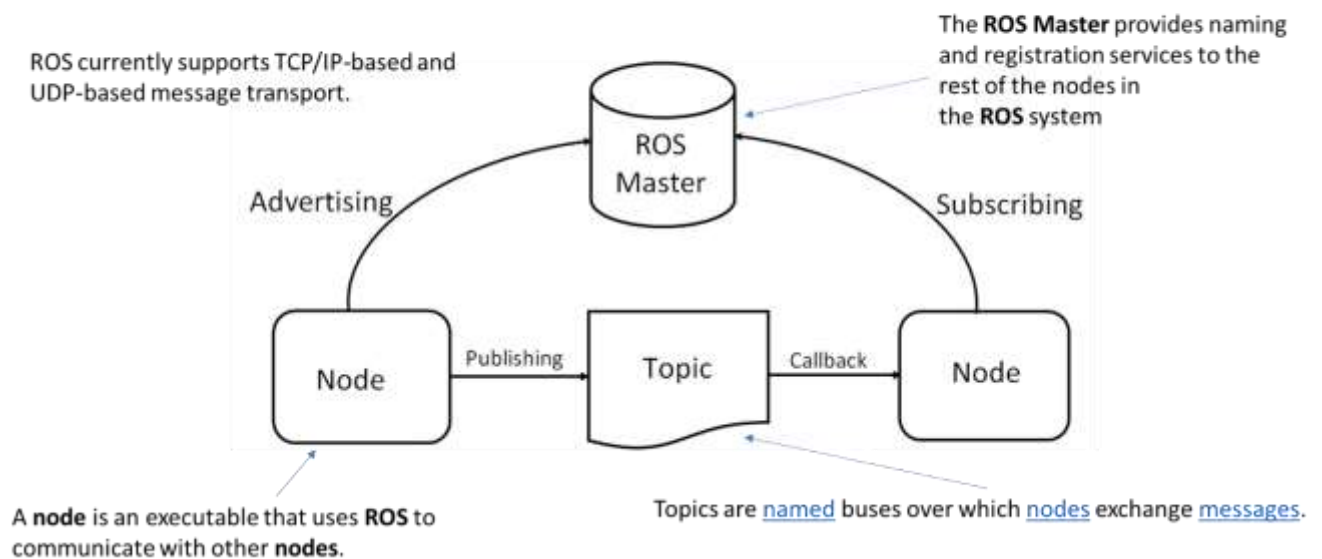
Documentation sourced from:

<http://wiki.ros.org/Documentation>

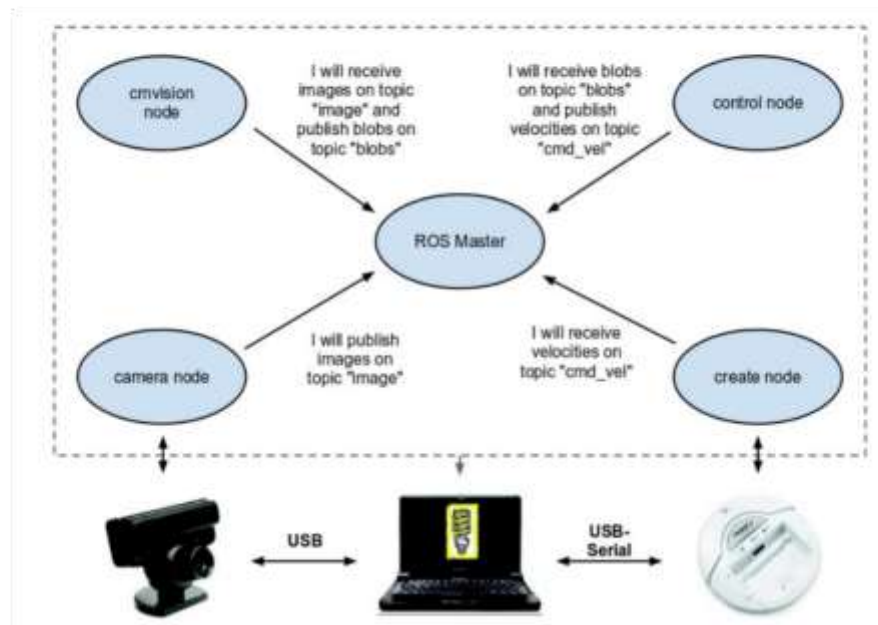
Quick Overview of Graph Concepts

- [Nodes](#): A node is an executable that uses ROS to communicate with other nodes.
- [Messages](#): ROS data type used when subscribing or publishing to a topic.
- [Topics](#): Nodes can *publish* messages to a topic as well as *subscribe* to a topic to receive messages.
- [Master](#): Name service for ROS (i.e. helps nodes find each other)
- [rosout](#): ROS equivalent of stdout/stderr
- [roscore](#): Master + rosout + parameter server (parameter server will be introduced later)

Working



Example of working:



Rosnode

A *node* is a process that performs computation. Nodes are combined together into a graph and communicate with one another using streaming [topics](#), RPC [services](#), and the [Parameter Server](#). These nodes are meant to operate at a fine-grained scale; a robot control system will usually comprise many nodes. For example, one node controls a laser range-finder, one Node controls the robot's wheel motors, one node performs localization, one node performs path planning, one node provides a graphical view of the system, and so on.

A ROS node is written with the use of a [ROS client library](#), such as [roscpp](#) or [rospy](#).

Node Tools

rostopic is a command-line tool for interacting with ROS nodes. For example:

```
$ rostopic list
```

will list the current topics and

```
$ rostopic echo /node_name
```

```
$ rostopic info /node_name
```

will display Messages published to `/topic_name`. See the [rostopic](#) page for more documentation.

Ros Message

[Nodes](#) communicate with each other by publishing **messages** to [topics](#). A message is a simple data structure, comprising typed fields. Standard primitive types (integer, floating point, boolean, etc.) are supported, as are arrays of primitive types. Messages can include arbitrarily nested structures and arrays (much like C structs).

Nodes can also exchange a *request* and *response* message as part of a ROS [service](#) call. These request and response messages are defined in [srv files](#).

msg files

[msg](#) files are simple text files for specifying the data structure of a message. These files are stored in the `msg` subdirectory of a package. For more information about these files, including a type specification, see the [msg format](#).

These Messages will come in Handy for the problem!!

E.g. Message

Ackermann Drive Stamped Messages

[ackermann_msgs/AckermannDriveStamped](#) Message

File: `ackermann_msgs/AckermannDriveStamped.msg`

Raw Message Definition

```
## Time stamped drive command for robots with Ackermann steering.
# $Id$

Header          header
AckermannDrive  drive
```

Compact Message Definition

```
std_msgs/Header header
ackermann_msgs/AckermannDrive drive
```

Odometry

http://docs.ros.org/melodic/api/nav_msgs/html/msg/Odometry.html

NAV_MSGS

http://docs.ros.org/melodic/api/nav_msgs/html/msg/Path.html

Message Types

Message types use standard ROS [naming](#) conventions: the name of the package + / + name of the .msg file. For example, std_msgs/msg/String.msg has the message type std_msgs/String.

In addition to the message type, messages are versioned by an MD5 sum of the .msg file. Nodes can only communicate messages for which both the message type and MD5 sum match.

Header

A message may include a special message type called 'Header', which includes some common metadata fields such as a timestamp and a frame ID. The ROS [Client Libraries](#) will automatically set some of these fields for you if you wish, so their use is highly encouraged.

There are three fields in the header message shown below. The `seq` field corresponds to an id that automatically increases as messages are sent from a given publisher. The `stamp` field stores time information that should be associated with data in a message. In the case of a laser scan, for example, the stamp might correspond to the time at which the scan was taken. The `frame_id` field stores frame information that should be associated with data in a message. In the case of a laser scan, this would be set to the frame in which the scan was taken.

```
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.
#
# sequence ID: consecutively increasing ID
uint32 seq

#Two-integer timestamp that is expressed as:
# * stamp.sec: seconds (stamp_secs) since epoch (in Python the variable is called 'secs')
# * stamp.nsec: nanoseconds since stamp_secs (in Python the variable is called 'nsecs')
# time-handling sugar is provided by the client library
time stamp

#Frame this data is associated with
```

```
# 0: no frame
# 1: global frame
string frame_id
```

Rostopic

Topics are [named](#) buses over which [nodes](#) exchange [messages](#). Topics have anonymous publish/subscribe semantics, which decouples the production of information from its consumption. In general, nodes are not aware of who they are communicating with. Instead, nodes that are interested in data *subscribe* to the relevant topic; nodes that generate data *publish* to the relevant topic. There can be multiple publishers and subscribers to a topic.

Topics are intended for unidirectional, streaming communication. Nodes that need to perform remote procedure calls, i.e. receive a response to a request, should use [services](#) instead. There is also the [Parameter Server](#) for maintaining small amounts of state.

Topic Types

Each topic is strongly typed by the ROS [message](#) type used to publish to it and nodes can only receive messages with a matching type. The [Master](#) does not enforce type consistency among the publishers, but subscribers will not establish message transport unless the types match. Furthermore, all ROS clients check to make sure that an MD5 computed from the [msg files](#) match. This check ensures that the ROS Nodes were compiled from consistent code bases.

Topic Transports

ROS currently supports TCP/IP-based and UDP-based message transport. The TCP/IP-based transport is known as [TCPROS](#) and streams message data over persistent TCP/IP connections. TCPROS is the default transport used in ROS and is the only transport that client libraries are required to support. The UDP-based transport, which is known as [UDPROS](#) and is currently only supported in [roscpp](#), separates messages into UDP packets. UDPROS is a low-latency, lossy transport, so is best suited for tasks like teleoperation.

ROS nodes negotiate the desired transport at runtime. For example, if a node prefers UDPROS transport but the other Node does not support it, it can fallback on TCPROS transport. This negotiation model enables new transports to be added over time as compelling use cases arise.

Topic Tools

[rostopic](#) is a command-line tool for interacting with ROS topics. For example:

```
$ rostopic list
```

will list the current topics and

```
$ rostopic echo /topic_name
```

will display Messages published to `/topic_name`. See the [rostopic](#) page for more documentation.

Rospy

Publishing to a topic

See also: [rospy.Publisher Code API](#)

You can create a handle to publish messages to a topic using the `rospy.Publisher` class. The most common usage for this is to provide the name of the topic and the message class/type of the topic. You can then call `publish()` on that handle to publish a message, e.g.:

```
pub = rospy.Publisher('topic_name', std_msgs.msg.String, queue_size=10)
pub.publish(std_msgs.msg.String("foo"))
```

rospy.Publisher initialization

```
rospy.Publisher(topic_name, msg_class, queue_size)
```

- The only required arguments to create a `rospy.Publisher` are the topic name, the `Message` class, and the `queue_size`. Note: `queue_size` is only available in Hydro and newer. e.g.

```
pub = rospy.Publisher('topic_name', std_msgs.msg.String, queue_size=10)
```

Publisher.publish()

There are three different ways of calling `publish()` ranging from an explicit style, where you provide a `Message` instance, to two implicit styles that create the `Message` instance on the fly.

Explicit style

- The explicit style is simple: you create your own `Message` instance and pass it to `publish`, e.g.:

```
pub.publish(std_msgs.msg.String("hello world"))
```

Complete example

```
1 import rospy
2 from std_msgs.msg import String
3
4 pub = rospy.Publisher('topic_name', String, queue_size=10)
5 rospy.init_node('node_name')
```

```
6 r = rospy.Rate(10) # 10hz
7 while not rospy.is_shutdown():
8     pub.publish("hello world")
9     r.sleep()
```

Subscribing to a topic

See also: [rospy.Subscriber Code API](#)

```
1 import rospy
2 from std_msgs.msg import String
3
4 def callback(data):
5     rospy.loginfo("I heard %s", data.data)
6
7 def listener():
8     rospy.init_node('node_name')
9     rospy.Subscriber("chatter", String, callback)
10    # spin() simply keeps python from exiting until this node is stopped
11    rospy.spin()
```

Connection Information

A subscriber can get access to a "connection header", which includes debugging information such as who sent the message, as well information like whether or not a message was latched. This data is stored as the `_connection_header` field of a received message.

e.g.

```
print m._connection_header
{'callerid': '/talker_38321_1284999593611',
 'latching': '0',
 'md5sum': '992ce8a1687cec8c8bd883ec73ca41d1',
 'message_definition': 'string data\n\n',
 'topic': '/chatter',
 'type': 'std_msgs/String'}
```


Writing the Publisher Node

"Node" is the ROS term for an executable that is connected to the ROS network. Here we'll create the publisher ("talker") node which will continually broadcast a message.

The Code

Make a Python file in the gedit software (search for gedit) and enter the following text below. Then save the file as **talker.py**

The file can be run by typing

python /path_to/talker.py

```
#!/usr/bin/env python
# license removed for brevity
import rospy
from std_msgs.msg import String

def talker():
    pub = rospy.Publisher('chatter', String, queue_size=10)
    rospy.init_node('talker', anonymous=True)
    rate = rospy.Rate(10) # 10hz
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()

if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

The Code Explained

Now, let's break the code down.

```
#!/usr/bin/env python
```

Every Python ROS [Node](#) will have this declaration at the top. The first line makes sure your script is executed as a Python script.

```
import rospy
from std_msgs.msg import String
```

You need to import `rospy` if you are writing a ROS [Node](#). The `std_msgs.msg` import is so that we can reuse the `std_msgs/String` message type (a simple string container) for publishing.

```
pub = rospy.Publisher('chatter', String, queue_size=10)
rospy.init_node('talker', anonymous=True)
```

This section of code defines the talker's interface to the rest of

ROS. `pub = rospy.Publisher("chatter", String, queue_size=10)` declares that your node is publishing to the `chatter` topic using the message type `String`. `String` here is actually the class `std_msgs.msg.String`. The `queue_size` argument limits the amount of queued messages if any subscriber is not receiving them fast enough. In older ROS distributions just omit the argument.

The next line, `rospy.init_node(NAME, ...)`, is very important as it tells `rospy` the name of your node -- until `rospy` has this information, it cannot start communicating with the ROS [Master](#). In this case, your node will take on the name `talker`. NOTE: the name must be a [base name](#), i.e. it cannot contain any slashes `"/"`.

`anonymous = True` ensures that your node has a unique name by adding random numbers to the end of `NAME`. Refer to [Initialization and Shutdown - Initializing your ROS Node](#) in the `rospy` documentation for more information about node initialization options.

```
rate = rospy.Rate(10) # 10hz
```

This line creates a `Rate` object `rate`. With the help of its method `sleep()`, it offers a convenient way for looping at the desired rate. With its argument of `10`, we should expect to go through the loop 10 times per second (as long as our processing time does not exceed 1/10th of a second!)

```
while not rospy.is_shutdown():
    hello_str = "hello world %s" % rospy.get_time()
    rospy.loginfo(hello_str)
    pub.publish(hello_str)
    rate.sleep()
```

This loop is a fairly standard `rospy` construct: checking the `rospy.is_shutdown()` flag and then doing work. You have to check `is_shutdown()` to check if your program should exit (e.g. if there is a `Ctrl-C` or otherwise). In this case, the "work" is a call to `pub.publish(hello_str)` that publishes a string to our `chatter` topic. The loop calls `rate.sleep()`, which sleeps just long enough to maintain the desired rate through the loop.

(You may also run across `rospy.sleep()` which is similar to `time.sleep()` except that it works with simulated time as well (see [Clock](#)).)

This loop also calls `rospy.loginfo(str)`, which performs triple-duty: the messages get printed to screen, it gets written to the Node's log file, and it gets written to [rosout](#). [rosout](#) is a handy tool for debugging: you can pull up messages using [rqt_console](#) instead of having to find the console window with your Node's output.

`std_msgs.msg.String` is a very simple message type, so you may be wondering what it looks like to publish more complicated types. The general rule of thumb is that *constructor args are in the same order as in the .msg file*. You can also pass in no arguments and initialize the fields directly, e.g.

```
msg = String()
msg.data = str
```

or you can initialize some of the fields and leave the rest with default values:

```
String(data=str)
```

You may be wondering about the last little bit:

[Toggle line numbers](#)

```
try:
    talker()
except rospy.ROSInterruptException:
    pass
```

In addition to the standard Python `__main__` check, this catches a `rospy.ROSInterruptException` exception, which can be thrown by `rospy.sleep()` and `rospy.Rate.sleep()` methods when Ctrl-C is pressed or your Node is otherwise shutdown. The reason this exception is raised is so that you don't accidentally continue executing code after the `sleep()`.

Now we need to write a node to receive the messages.

Writing the Subscriber Node

The Code

Make a Python file in the gedit software (search for gedit) and enter the following text below. Then save the file as **listener.py**

The file can be run by typing

```
python /path_to/listener.py
```

The file contents look close to:

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)

def listener():

    # In ROS, nodes are uniquely named. If two nodes with the same
    # name are launched, the previous one is kicked off. The
    # anonymous=True flag means that rospy will choose a unique
    # name for our 'listener' node so that multiple listeners can
    # run simultaneously.
    rospy.init_node('listener', anonymous=True)

    rospy.Subscriber("chatter", String, callback)
```

```
# spin() simply keeps python from exiting until this node is stopped
rospy.spin()

if __name__ == '__main__':
    listener()
```

The Code Explained

The code for `listener.py` is similar to `talker.py`, except we've introduced a new callback-based mechanism for subscribing to messages.

[Toggle line numbers](#)

```
rospy.init_node('listener', anonymous=True)

rospy.Subscriber("chatter", String, callback)

# spin() simply keeps python from exiting until this node is stopped
rospy.spin()
```

This declares that your node subscribes to the `chatter` topic which is of type `std_msgs.msgs.String`. When new messages are received, `callback` is invoked with the message as the first argument.

We also changed up the call to `rospy.init_node()` somewhat. We've added the `anonymous=True` keyword argument. ROS requires that each node have a unique name. If a node with the same name comes up, it bumps the previous one. This is so that malfunctioning nodes can easily be kicked off the network. The `anonymous=True` flag tells `rospy` to generate a unique name for the node so that you can have multiple `listener.py` nodes run easily.

The final addition, `rospy.spin()` simply keeps your node from exiting until the node has been shutdown. Unlike `roscpp`, `rospy.spin()` does not affect the subscriber callback functions, as those have their own threads.